OBJECT-ORIENTED ANALYSIS AND DESIGN

Course Summary · 2025–2026

Vladimir Soroka

Shenkar · Software Engineering

What We Covered

01

Smart Pointers

unique_ptr · shared_ptr · weak_ptr

02

Reference Counting

Ownership, cycles, and memory management

03

UML Diagrams

9 diagram types across all SDLC stages

04

Software Design + SOLID

Core principles for maintainable architecture

05

Creational Patterns

Singleton · Prototype · Abstract Factory · Builder

06

Structural & Behavioral

Composite · Decorator · Proxy · Observer · Strategy · …

Code Design: Smart Pointers

Replaces raw pointers — automatic resource management, no manual delete

unique_ptr

Exclusive ownership

Sole owner of a resource. Cannot be copied — only moved. Automatically destroys the object when it goes out of scope. Zero runtime overhead vs. raw pointer.

Default choice when ownership is singular and clear.

shared_ptr

Shared ownership

Multiple owners share a resource via reference counting. The object is destroyed when the last shared_ptr is released. Slight overhead from atomic counter updates.

Use when multiple objects must share ownership.

weak_ptr

Non-owning observer

Observes a shared_ptr without incrementing the ref-count. Must be locked (→ shared_ptr) before use. Safely checks whether the observed object still exists.

Breaks circular references. Caches. Back-pointers.

Code Design: Reference Counting

How It Works

1. Each object stores an integer reference count
2. Count increments on copy or new assignment
3. Count decrements when a reference is released
4. Object is destroyed when count reaches zero

Key Considerations

Circular References

A → B → A: the ref-count never reaches zero, so neither object is freed. Solution: use weak_ptr for back-references.

Thread Safety

Count updates must be atomic in concurrent programs. shared_ptr uses atomic increment/decrement internally.

Performance Cost

Atomic operations add overhead. Prefer unique_ptr (zero overhead) when shared ownership is not needed.

UML Diagrams

When & how to use each diagram — at which stage, by whom

Structural (5)

Class	Core model: classes, attributes, methods, relationships
Object	Snapshot of specific instances at a point in time
Package	High-level module organization and dependencies
Component	System components and their provided/required interfaces
Deployment	Physical nodes and software distribution

Behavioral (5)

Use Case	System scope: actors and their goals
Sequence	Message flow between objects over time (focus: order)
Communication	Same as Sequence but emphasizes object links
Activity	Workflow and control flow — a detailed flowchart
State	Object lifecycle: states and event-driven transitions

UML: Class Diagram

Shows

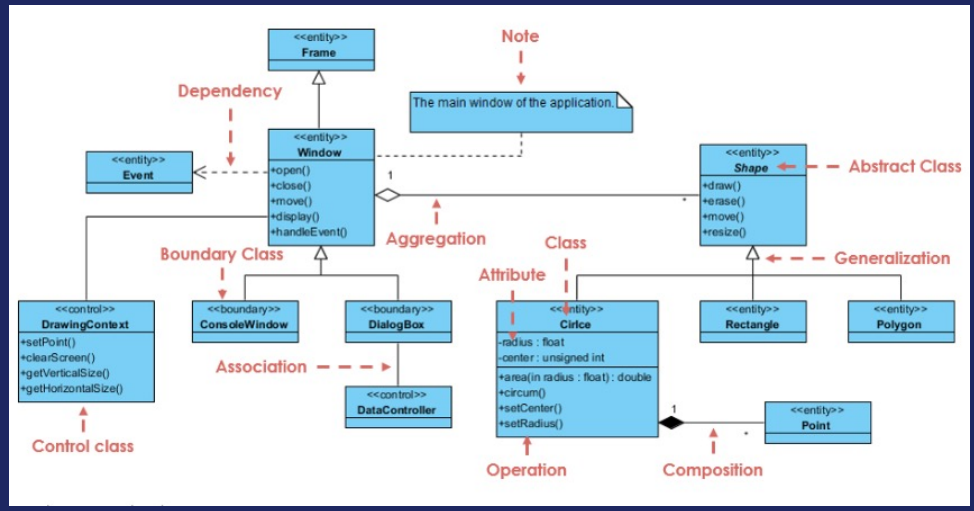
The structural blueprint — classes, their attributes, operations, and the relationships between them. The most widely used UML diagram.

Key elements

3-compartment box (name | attributes | methods). Relationships: inheritance △, association →, aggregation ◇–, composition ◆–, dependency --▶

Stage & role

Analysis & Design. Used by architects and developers throughout the project to model both the domain and the software structure.



UML: Object Diagram

Shows

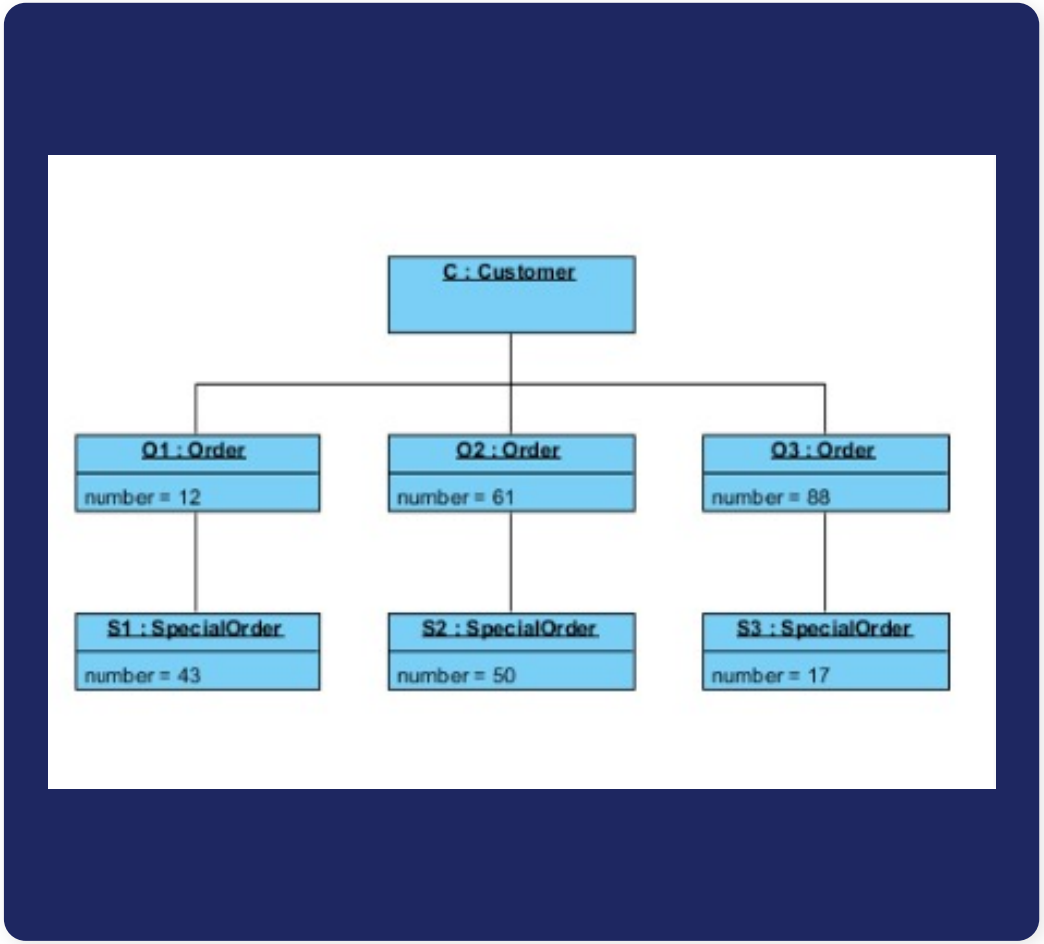
A snapshot of the system at a specific instant — actual object instances with concrete values, not class templates.

Key elements

Object box: instanceName : ClassName. Attribute values shown (not types). Links between objects (not associations). No method compartment.

Stage & role

Design & Testing. Developers use it to illustrate a concrete scenario, clarify an ambiguous class diagram, or debug object state.



UML: Package Diagram

Shows

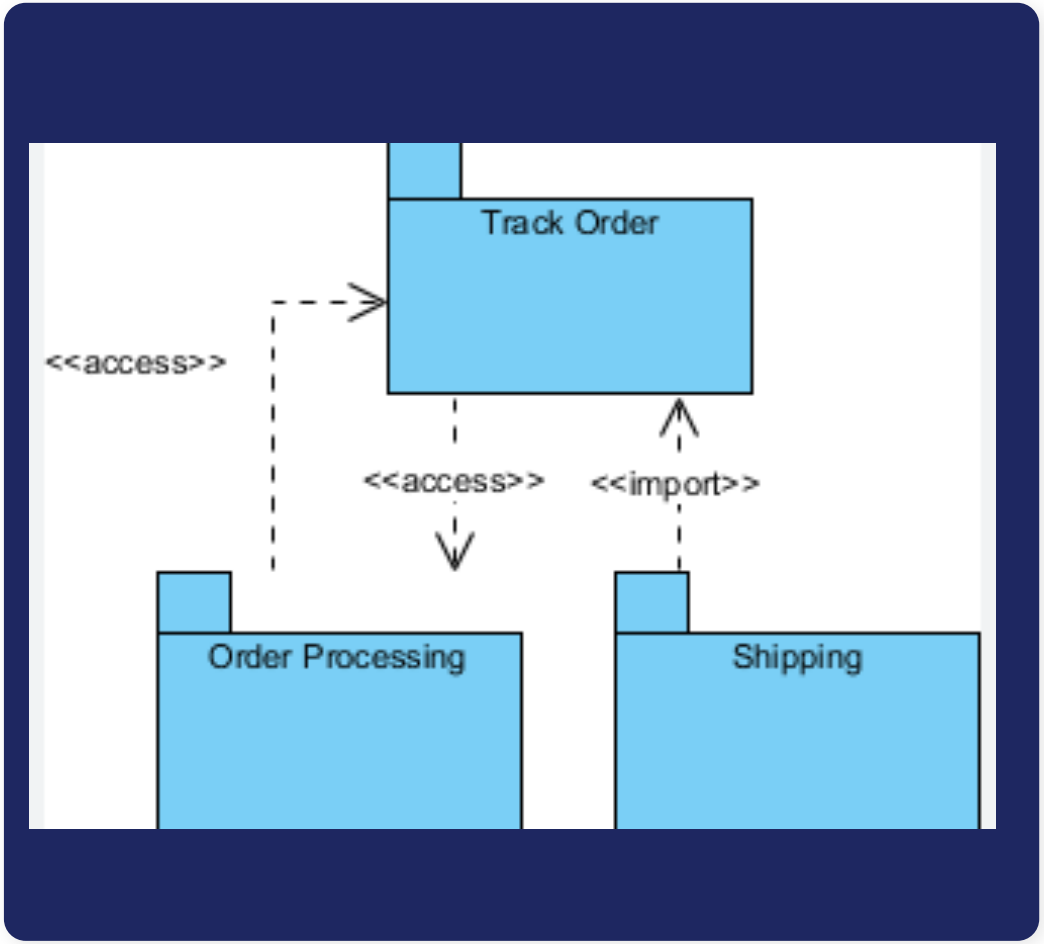
High-level modular structure — how the system is divided into packages (namespaces) and how they depend on each other.

Key elements

Package (tabbed rectangle). «use» / «import» dependency arrows. Contained elements (classes, subsystems). Enables layering and cycle detection.

Stage & role

Early Design. Architects define system layering, module boundaries, and allowed dependency directions before detailed design begins.



UML: Component Diagram

Shows

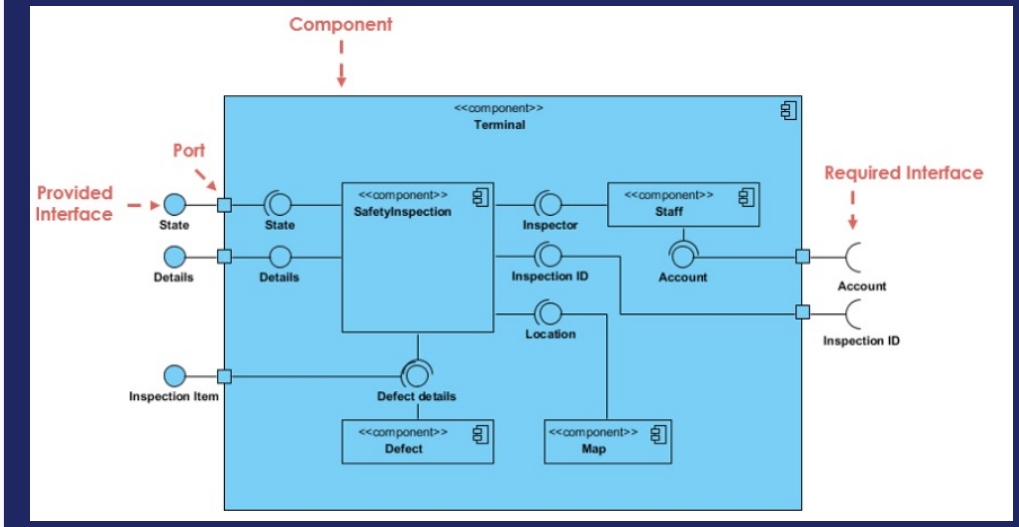
Logical or physical components and their interfaces. Shows how large system parts connect via provided and required interfaces.

Key elements

Component (box with «component»). Provided interface (lollipop –○). Required interface (socket –⊂). Dependency arrows between components.

Stage & role

Design phase. Architects decompose the system into replaceable components and specify their interaction contracts.



UML: Deployment Diagram

Shows

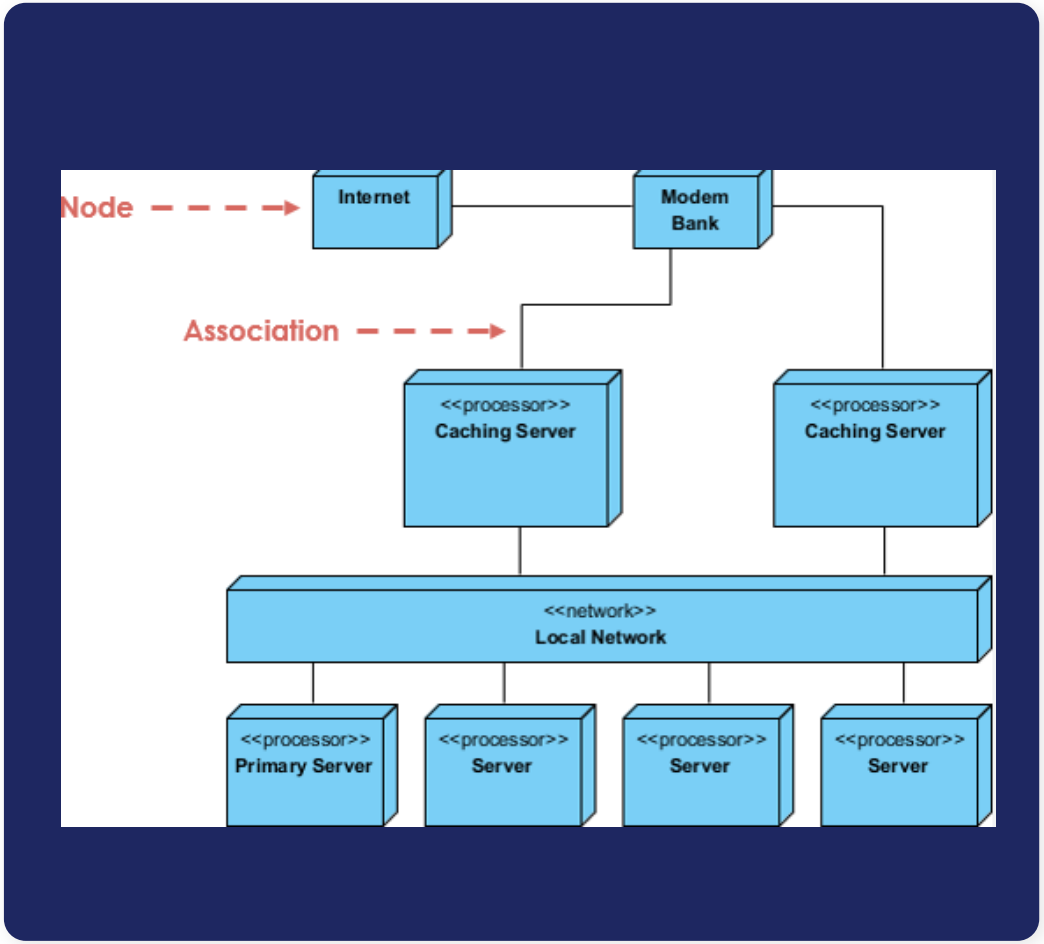
Physical distribution of software — which artifacts run on which hardware nodes, and the communication paths between them.

Key elements

Node/Device (rectangle with «device»). Artifact (deployed file or executable). Communication path (line + protocol label).

Stage & role

Late Design & Delivery. Architects, DevOps, and infrastructure teams plan deployment topology and runtime environments.



UML: Use Case Diagram

Shows

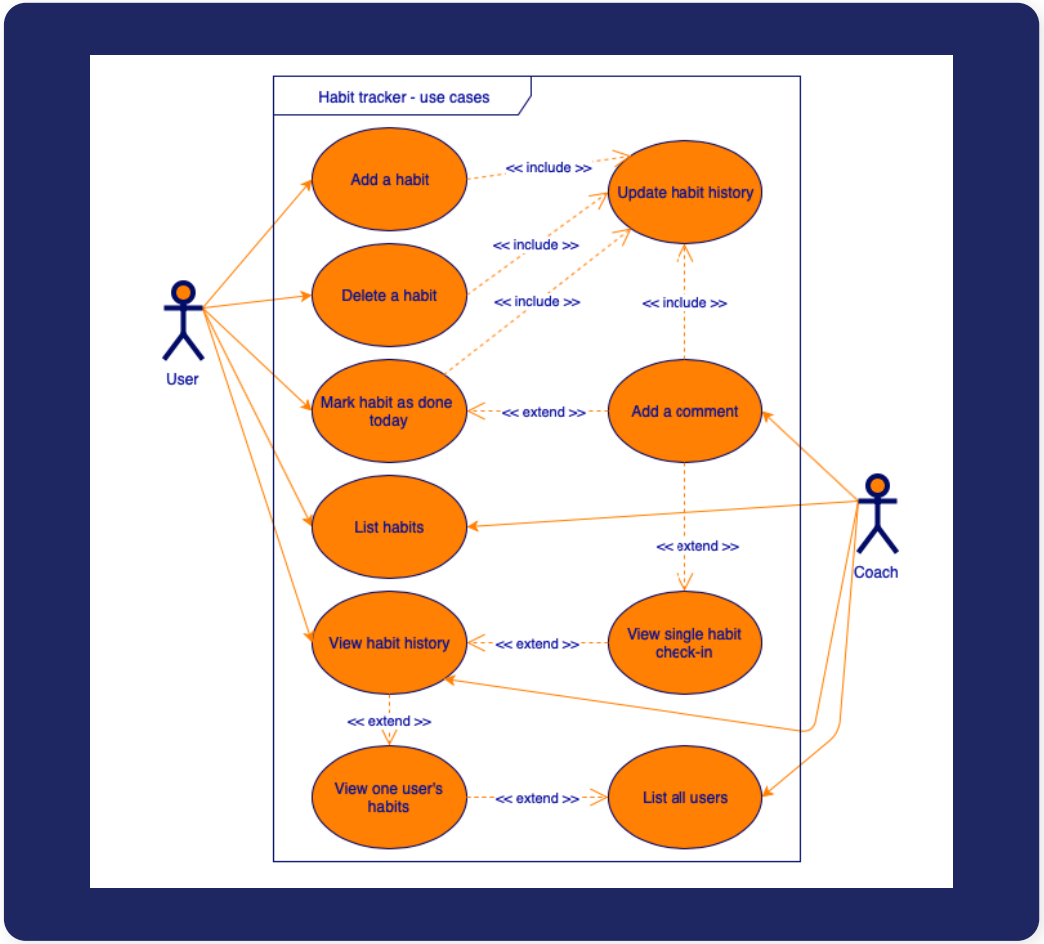
Functional scope from the user's perspective — what the system does, for whom. Defines the system boundary and all external actors.

Key elements

Actor (stick figure). Use case (ellipse). System boundary (rectangle). Relationships: association (–), «include», «extend».

Stage & role

Requirements & Analysis. Business analysts and stakeholders define scope; the diagram is the contract for what will be built.



UML: Sequence Diagram

Shows

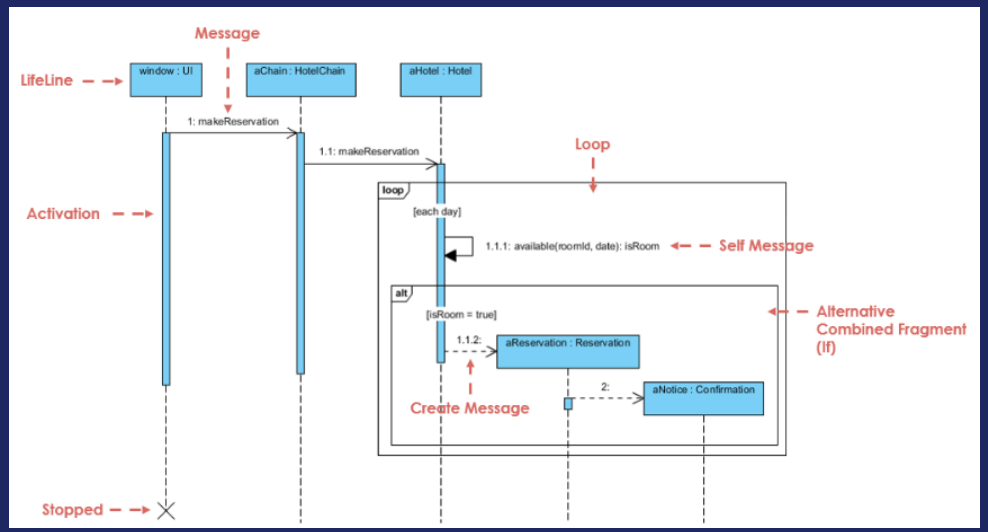
How objects interact over time — the exact order of messages exchanged between participants for a specific scenario.

Key elements

Lifeline (vertical dashed line). Activation bar (thin box on lifeline). Synchronous message (→). Return message (-->). Time flows downward.

Stage & role

Design phase. Developers and architects detail how objects collaborate for one particular use case or operation.



UML: Communication Diagram

Shows

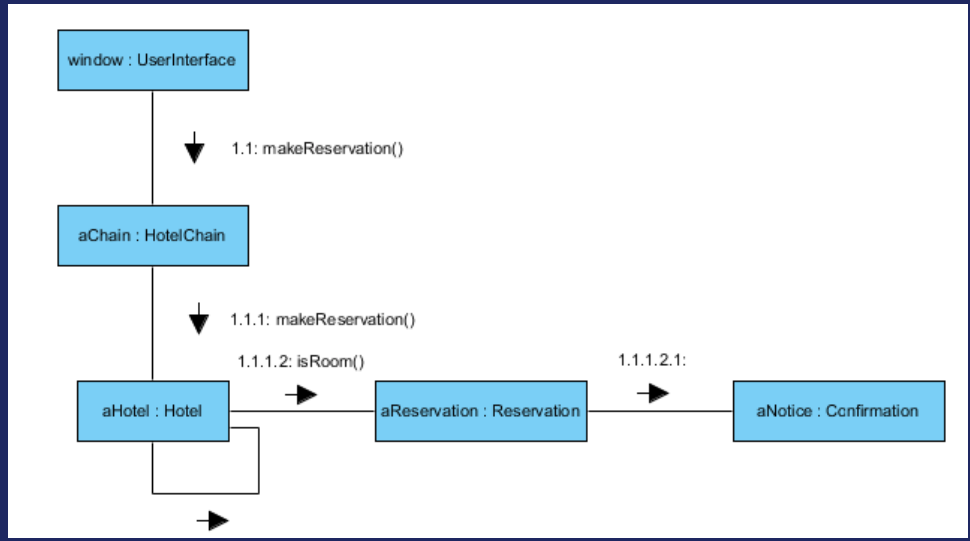
Same information as Sequence, but organized around object structure. Shows which objects link to which, with numbered messages.

Key elements

Objects (rectangles). Links between objects (solid lines). Numbered messages on links: 1:, 1.1:, 2: etc. showing call order.

Stage & role

Design phase. Preferred when the structure of collaborations matters more than the strict timeline of messages.



UML: Activity Diagram

Shows

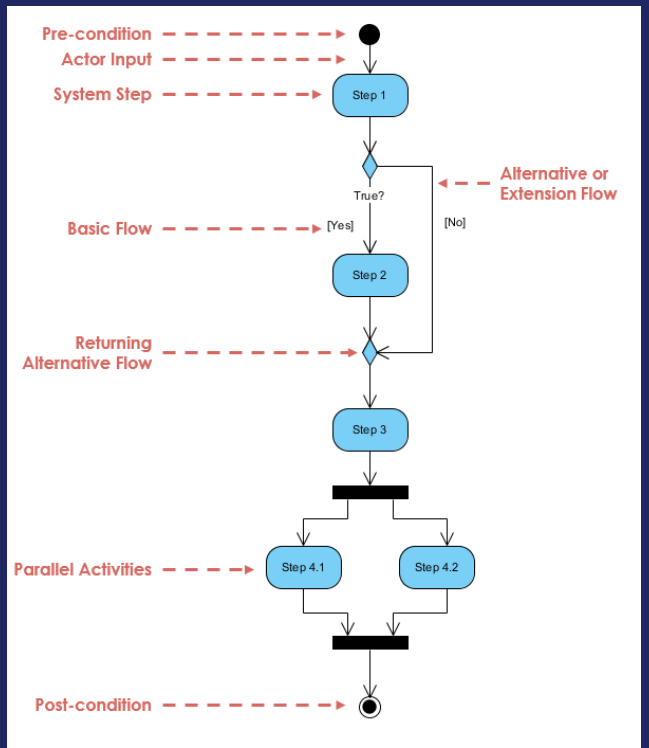
Workflow and control flow — actions, decisions, parallel activities, and flow between them. An advanced flowchart.

Key elements

Start (●), End (◉), Action (rounded rect), Decision (◇ with guards), Fork/Join (thick bar), Swimlanes for responsibility.

Stage & role

Analysis & Design. Business analysts model business processes; developers design algorithms and complex method logic.



UML: State Diagram

Shows

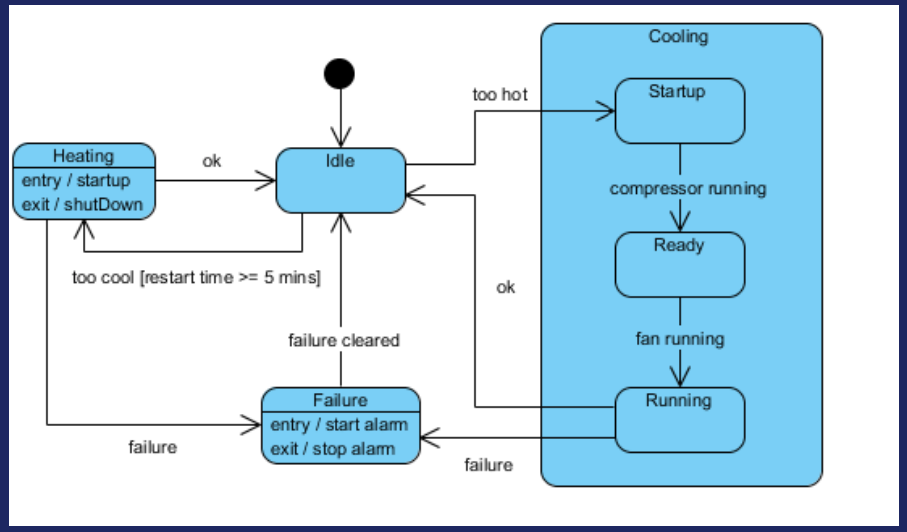
Lifecycle of a single object — all states it can be in and the events or conditions that trigger transitions between states.

Key elements

State (rounded rectangle). Transition (arrow labeled: event [guard] / action). Initial pseudostate (●). Final state (◉). Composite states.

Stage & role

Design phase. Used for objects with complex, state-dependent behavior: UI components, protocols, order lifecycles, hardware devices.



Principles of Software Design

SOLID — five principles for maintainable, flexible object-oriented design

S	**Single Responsibility**	A class should have one, and only one, reason to change.
O	**Open / Closed**	Open for extension, closed for modification.
L	**Liskov Substitution**	Subtypes must be substitutable for their base types without altering program correctness.
I	**Interface Segregation**	Prefer many small, focused interfaces over one large general-purpose one.
D	**Dependency Inversion**	Depend on abstractions, not on concretions. High-level modules should not depend on low-level modules.

These principles guide design decisions — applied before and alongside selecting a specific design pattern.

Design Patterns

Reusable solutions to common software design problems (Gang of Four — Gamma et al., 1995)

Creational

How objects are created

- Singleton
- Prototype
- Abstract Factory
- Builder

Structural

How objects are composed

- Composite
- Decorator
- Proxy
- Adapter
- Facade

Behavioral

How objects communicate

- Observer
- Template Method
- Strategy
- Iterator
- Command

13 patterns total — each solving a specific, recurring design problem

Creational Patterns

Singleton

Ensure only one instance of a class exists and provide global access to it.

e.g. Config manager, Logger, Thread pool

Prototype

Create new objects by cloning an existing object rather than constructing from scratch.

e.g. Costly initialization, avoiding subclassing for variations

Abstract Factory

Provide an interface for creating families of related objects without specifying concrete classes.

e.g. Cross-platform UI toolkit (Button, Checkbox per OS)

Builder

Separate the step-by-step construction of a complex object from its final representation.

e.g. Document generator, query builder, complex config objects

Structural Patterns

Composite

Compose objects into tree structures. Clients treat individual objects and compositions uniformly.

e.g. File system, UI trees, org charts

Decorator

Attach responsibilities to an object dynamically. Flexible alternative to subclassing.

e.g. I/O streams, GUI widgets with borders/scroll

Proxy

Provide a surrogate for another object to control access, enable lazy loading, caching, or logging.

e.g. Virtual proxy, protection proxy, remote proxy

Adapter

Convert the interface of a class into another interface that clients expect. Lets incompatible interfaces work together.

e.g. Wrapping a legacy API, adapting third-party libraries to a local interface

Facade

Provide a simplified interface to a complex subsystem. Reduces coupling between clients and subsystem internals.

e.g. Compiler front-end, home theatre controller, unified API over multiple services

Behavioral Patterns

Observer

One-to-many dependency: when one object changes state, all its dependents are notified automatically.

e.g. Event systems, MVC notifications, pub/sub

Template Method

Define the skeleton of an algorithm in a base class; subclasses fill in specific steps.

e.g. Frameworks, game loops, data import pipelines

Strategy

Define a family of algorithms, encapsulate each one, and make them interchangeable at runtime.

e.g. Sorting, payment methods, compression strategies

Iterator

Provide a uniform way to access elements of a collection sequentially without exposing its internal representation.

e.g. STL iterators, range-based for loops, database cursors

Command

Encapsulate a request as an object, letting you parameterize clients, queue or log requests, and support undoable operations.

e.g. GUI actions (undo/redo), task queues, macro recording

Course Complete

From smart pointers to design patterns — you now have the tools to design and analyze complex systems

1

Analyze First

Requirements → system scope before any design

2

Design Before Code

UML + principles → clean, maintainable implementation

3

Patterns Are Tools

Know when to apply them — not every problem needs one

Good luck on the exam! · Vladimir Soroka